New complementarity formulations for nonsmooth process models

Maha Chaudhry Yulan Zhang Kamil A. Khan
kamilkhan@mcmaster.ca

Department of Chemical Engineering, McMaster University

AIChE, Minneapolis

Nov 9, 2026

Outline

Motivation and background
- Challenges of nonsmoothness
- Neural Networks
- Abs-normal forms
- Complementary Problems

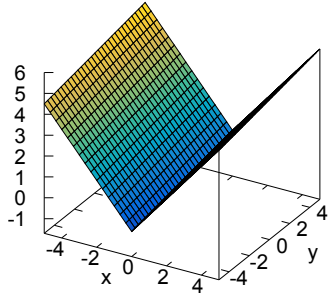
New results concerning piecewise-affine functions
- Root finding for ReLU neural networks
- Root finding for neural ODEs
- New root-finding and optimization approaches
- Implementation and examples

Challenges of nonsmoothness

Numerical methods for smooth systems may perform poorly or fail when nonsmoothness is encountered

Descent directions become difficult to find

- Negative of a subgradient is not necessarily a descent direction
- Example: $f(x, y) = \max(0.1y + x,\ 0.1y - x)$



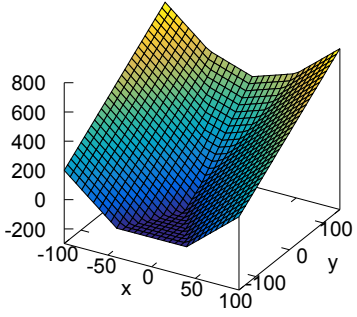
Challenges of nonsmoothness

Numerical methods for smooth systems may perform poorly or fail when nonsmoothness is encountered

Reliable methods may fail

Example from Hiriart-Urruty and Lemaréchal (1993):

- $f(x, y) = \max(-100,\ 3y \pm 2x,\ 2y \pm 5x)$



- Failure of steepest descent with exact line search

Nonsmoothness in applications

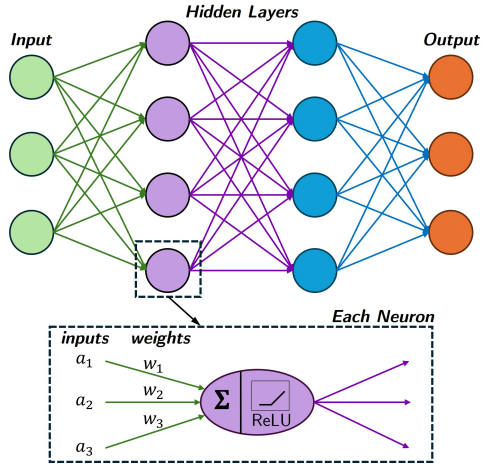
- Modeling discrete-continuous phenomena
  - thermodynamic phase changes
  - multi-stage processes (e.g. pressure-swing adsorption)
- Activation of safety mechanisms
  - check valves, relief valves
- Embedded optimization problems
  - dynamic flux balance analysis (Hjersted and Henson, 2009)
  - closed-form optimal control
- Aggressive penalty methods for constrained optimization
- Neural networks (e.g., ReLU activation)

Neural Network Architecture

From Aydinoglu et al. [2021], a NN $\phi(\mathbf{x})$ is a multilayer perceptron with L hidden layers:

$$\phi(\mathbf{x}) = (\mathbf{h}^{(L+1)} \circ \boldsymbol{\lambda}_{ReLU} \circ \mathbf{h}^{(L)} \circ ... \boldsymbol{\lambda}_{ReLU} \circ \mathbf{h}^{(1)})(\mathbf{x})$$

- **Affine function** ($\mathbf{h}^{(\ell)}$) - describing weighted linear relationship between neurons between hidden layers. It satisfies: $\mathbf{h}^{(\ell)}(\mathbf{x}) = \boldsymbol{\theta}^{(\ell)}\mathbf{x} + \mathbf{C}^{(\ell)}$
- **Activation function** (λ_{ReLU}) satisfies $\boldsymbol{\lambda}_{ReLU}(\mathbf{x}) = \max(\mathbf{0}, \mathbf{x})$



Aydinoglu, A., Fazlyab, M., Morari, M., & Posa, M. Stability Analysis of Complementarity Systems with Neural Network Controllers, ACM. 2021; 19: 1-10.

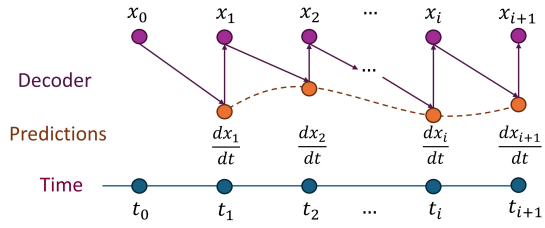
Neural ODEs

Definition

- A neural network ϕ describes the time-derivatives of the state variables [Chen et al. [2018]].

$$\frac{dx}{dt}(t) = \phi(x(t))$$

- Computed using finite-step discrete approximations like the Euler method.



Abs-normal forms

Griewank et al., *Linear Algebra Appl.* (2015)

Definition

Given a continuous piecewise-affine function:

$$\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m,$$

there exist matrices:

$$\mathbf{Z} \in \mathbb{R}^{s \times n}, \quad \mathbf{J} \in \mathbb{R}^{m \times n}, \quad \mathbf{Y} \in \mathbb{R}^{m \times s}, \quad \mathbf{L} \in \mathbb{R}^{s \times s} \text{ (strictly lower triangular)},$$

and vectors:

$$\mathbf{c} \in \mathbb{R}^s, \quad \mathbf{b} \in \mathbb{R}^m,$$

such that, for any $\mathbf{x} \in \mathbb{R}^n$, there is a unique vector $\mathbf{z} \in \mathbb{R}^s$ satisfying:

$$\begin{bmatrix} \mathbf{z} \\ \mathbf{f}(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \mathbf{c} \\ \mathbf{b} \end{bmatrix} + \begin{bmatrix} \mathbf{Z} & \mathbf{L} \\ \mathbf{J} & \mathbf{Y} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ |\mathbf{z}| \end{bmatrix}.$$

This representation is an abs-normal form of f.

Observations

- $\mathbf{z}$'s components are the arguments of the absolute value operations in f.

Construction example

- A forward automatic-differentiation like procedure
- Consider the piecewise-affine mapping:

$$f(x) = x + \Big|2|3x + 4| - 5\Big| + 6|7x - 8|,$$

- Construct the abs-normal form for f:
  1. Define a vector $\mathbf{z} \in \mathbb{R}^s$ for which:

$$\begin{aligned} z_1 &:= 3x + 4, \\ z_2 &:= 2|3x + 4| - 5 = 2|z_1| - 5, \\ z_3 &:= 7x - 8. \end{aligned}$$

  2. Express f in terms of x and $|\mathbf{z}|$:

$$f(x) = x + |z_2| + 6|z_3|.$$

  3. The abs-normal form for f is:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ f(x) \end{bmatrix} = \begin{bmatrix} 4 \\ -5 \\ -8 \\ 0 \end{bmatrix} + \begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 7 & 0 & 0 & 0 \\ 1 & 0 & 1 & 6 \end{bmatrix} \begin{bmatrix} x \\ |z_1| \\ |z_2| \\ |z_3| \end{bmatrix}.$$

Construction example

- A forward automatic-differentiation like procedure
- Consider the piecewise-affine mapping:

$$f(x) = x + \Big|2|3x + 4| - 5\Big| + 6|7x - 8|,$$

- Construct the abs-normal form for f:
  1. Define a vector $\mathbf{z} \in \mathbb{R}^s$ for which:

$$\begin{aligned} z_1 &:= 3x + 4, \\ z_2 &:= 2|3x + 4| - 5 = 2|z_1| - 5, \\ z_3 &:= 7x - 8. \end{aligned}$$

  2. Express f in terms of x and $|\mathbf{z}|$:

$$f(x) = x + |z_2| + 6|z_3|.$$

  3. The abs-normal form for f is:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ f(x) \end{bmatrix} = \begin{bmatrix} 4 \\ -5 \\ -8 \\ 0 \end{bmatrix} + \begin{bmatrix} 3 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 7 & 0 & 0 & 0 \\ 1 & 0 & 1 & 6 \end{bmatrix} \begin{bmatrix} x \\ |z_1| \\ |z_2| \\ |z_3| \end{bmatrix}.$$

$$\begin{bmatrix} \mathbf{z} \\ \mathbf{f(x)} \end{bmatrix} = \begin{bmatrix} \mathbf{c} \\ \mathbf{b} \end{bmatrix} + \begin{bmatrix} \mathbf{Z} & \mathbf{L} \\ \mathbf{J} & \mathbf{Y} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ |\mathbf{z}| \end{bmatrix}.$$

Construction for Neural Network

(In preparation)

ReLU NNs are piecewise affine composite functions. Consolidate into a sequence of intermediate variables $\{z^{(\ell)} \mid \ell \in R^L\}$ for abs-normal form mapping.

Piecewise affine mapping to abs-normal form for ReLU NN

Partition NN parameters into abs-normal form

ReLU NN

$$
\begin{aligned}
z^{(1)}(x) &= \Theta^{(1)} x && + c^{(1)} \\
z^{(2)}(x) &= \Theta^{(2)}\left(\lambda z^{(1)} + (1-\lambda)\left|z^{(1)}\right|\right) && + c^{(2)} \\
z^{(3)}(x) &= \Theta^{(3)}\left(\lambda z^{(2)} + (1-\lambda)\left|z^{(2)}\right|\right) && + c^{(3)} \\
&\vdots \\
\phi(x) &= \Theta^{(\ell)}\left(\lambda z^{(\ell-1)} + (1-\lambda)\left|z^{(\ell-1)}\right|\right) && + c^{(\ell)}
\end{aligned}
$$

$$
\begin{bmatrix} z \\ \phi(x) \end{bmatrix} = \begin{bmatrix} Z & L \\ J & Y \end{bmatrix} \begin{bmatrix} x \\ |z| \end{bmatrix} + \begin{bmatrix} c \\ b \end{bmatrix}
$$

Goal: Abs-normal form

Abs-normal matrix set-up

$$
\begin{bmatrix} z^{(1)} \\ z^{(2)} \\ \vdots \\ \phi \end{bmatrix} = \begin{bmatrix} \Theta^{(1)} & 0 & \dots & 0 \\ \lambda\Theta^{(2)}\Theta^{(1)} & (1-\lambda)\Theta^{(2)} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ \lambda^{\ell-1}\prod_{i=1}^{\ell}\Theta^{(i)} & \lambda^{\ell-2}(1-\lambda)\prod_{i=2}^{\ell}\Theta^{(i)} & \dots & (1-\lambda)\Theta^{(\ell)} \end{bmatrix} \begin{bmatrix} x \\ |z^{(1)}| \\ \vdots \\ |z^{(\ell-1)}| \end{bmatrix} + \begin{bmatrix} c^{(1)} \\ \lambda\Theta^{(2)}c^{(1)} + c^{(2)} \\ \vdots \\ \sum_{i=1}^{\ell}\lambda^{\ell-i}\left(\prod_{j=i+1}^{\ell}\Theta^{(j)}\right)c^{(i)} \end{bmatrix}
$$

Construction for Neural Network

(In preparation)

Calculating abs-normal form coefficients

- Griewank & Walther (2018) describes a similar formulation for ReLU NNs which relies on the abs-linear form.
- Formula was (i) adapted to abs-normal form and (ii) to include NN bias parameters and alternative activation functions.

$$\mathbf{M}\,\mathbf{z} = \zeta\begin{bmatrix}\mathbf{x}\\ |\mathbf{z}|\end{bmatrix} + \mathbf{C}$$

$$\begin{bmatrix} I & 0 & \cdots & 0 & 0 \\ -\lambda\Theta^{(2)} & I & \cdots & 0 & 0 \\ 0 & -\lambda\Theta^{(3)} & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & I & 0 \\ 0 & 0 & \cdots & -\lambda\Theta^{(s+1)} & I \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ \vdots \\ z_s \\ z_{s+1} \end{bmatrix} = \begin{bmatrix} \Theta^{(1)} & 0 & 0 & \cdots & 0 \\ 0 & (1-\lambda)\Theta^{(2)} & 0 & \cdots & 0 \\ 0 & 0 & (1-\lambda)\Theta^{(3)} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & 0 & \cdots & (1-\lambda)\Theta^{(s+1)} \end{bmatrix} \begin{bmatrix} |z_1| \\ |z_2| \\ |z_3| \\ \vdots \\ |z_s| \\ |z_{s+1}| \end{bmatrix} + \begin{bmatrix} \mathbf{C}^{(1)} \\ \mathbf{C}^{(2)} \\ \mathbf{C}^{(3)} \\ \vdots \\ \mathbf{C}^{(s)} \\ \mathbf{C}^{(s+1)} \end{bmatrix}$$

where $\lambda = \frac{1}{2}$ for regular ReLU and $\lambda = \frac{1+\alpha}{2}$ for leaky ReLU.

Complementary Problems

- Variables x and y are **complementary** to each other if the following hold:
  1. $\mathbf{x} \geq \mathbf{0}$
  2. $\mathbf{y} \geq \mathbf{0}$
  3. $\mathbf{x}^T\mathbf{y} = \mathbf{0}$
- **Notation**: Written compactly as $\mathbf{0} \leq \mathbf{x} \perp \mathbf{y} \geq \mathbf{0}$.
- **Linear Complementary Problem** (LCP): Find $\mathbf{x}$ such that:

$$\mathbf{0} \leq \mathbf{x} \perp \mathbf{A}\mathbf{x} + \mathbf{b} \geq \mathbf{0}$$

- **Mixed Linear Complementary Problem** (MLCP): For problems with both (i) complementary variable $\mathbf{x}$ and (ii) free variable $\mathbf{z}$:

$$\begin{aligned} \mathbf{A}\mathbf{x} - \mathbf{B}\mathbf{z} + \mathbf{a} &= \mathbf{0} \\ \mathbf{0} \leq \mathbf{x} \perp \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{z} + \mathbf{b} &\geq \mathbf{0} \end{aligned}$$

Outline

Motivation and background
- Challenges of nonsmoothness
- Neural Networks
- Abs-normal forms
- Complementary Problems

New results concerning piecewise-affine functions
- Root finding for ReLU neural networks
- Root finding for neural ODEs
- New root-finding and optimization approaches
- Implementation and examples

Root finding for ReLU neural networks

(In preparation)

- **Goal**: Given a fitted ReLU neural network ϕ, solve the following for $\mathbf{x}$:

$$\boldsymbol{g}(\mathbf{x}, \boldsymbol{\phi}(\mathbf{x}, p)) = 0$$

- Where residual function $\boldsymbol{g} : (R^n \times R^n) \to R^n$ is defined as:

$$\boldsymbol{g}(\mathbf{x}, \mathbf{y}) = \mathbf{P}\mathbf{x} + \mathbf{Q}\mathbf{y} + \mathbf{r}$$

Problem Cases

- **Case 1A**: Calculate root of neural network - i.e., solve $\phi(\mathbf{x}, p) = 0$.
  Set $\mathbf{P} = \mathbf{0}$, $\mathbf{Q} = \mathbf{I}$, *and* $\mathbf{r} = \mathbf{0}$.
- **Case 1B**: Inverse problem of neural network - i.e., solve $\phi(\mathbf{x}, p) = \mathbf{r}$.
  Set $\mathbf{P} = \mathbf{0}$, $\mathbf{Q} = \mathbf{I}$, *and user-select* $\mathbf{r}$.
- **Case 2**: Solve 1st-order neural ODE using Implicit Euler equation.
  Set $\mathbf{P} = \mathbf{I}$, $\mathbf{Q} = \Delta t\, \mathbf{I}$, *and* $\mathbf{r} = -\mathbf{x}^{(\mathbf{k})}$.

Root finding for ReLU neural networks

(In preparation)

- **Goal**: Given a fitted ReLU neural network ϕ, solve the following for $\mathbf{x}$:

$$\boldsymbol{g}(\mathbf{x}, \phi(\mathbf{x}, p)) = 0$$

- Where residual function $\boldsymbol{g} : (R^n \times R^n) \to R^n$ is defined as:

$$\boldsymbol{g}(\mathbf{x}, \mathbf{y}) = \mathbf{P}\mathbf{x} + \mathbf{Q}\mathbf{y} + \mathbf{r}$$

Summary of Approach

1. ReLU NNs are piecewise affine composite functions
2. Formulate abs-normal representation to solve $\boldsymbol{g}(\mathbf{x}, \phi(\mathbf{x}, p)) = \mathbf{0}$.

$$\begin{bmatrix} \mathbf{z} \\ \boldsymbol{g}(\mathbf{x}, \phi) \end{bmatrix} = \begin{bmatrix} \mathbf{c} \\ \overline{\mathbf{b}} \end{bmatrix} + \begin{bmatrix} \mathbf{Z} & \mathbf{L} \\ \overline{\mathbf{J}} & \overline{\mathbf{Y}} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ |\mathbf{z}| \end{bmatrix}$$

   where the following matrices and vectors are defined:

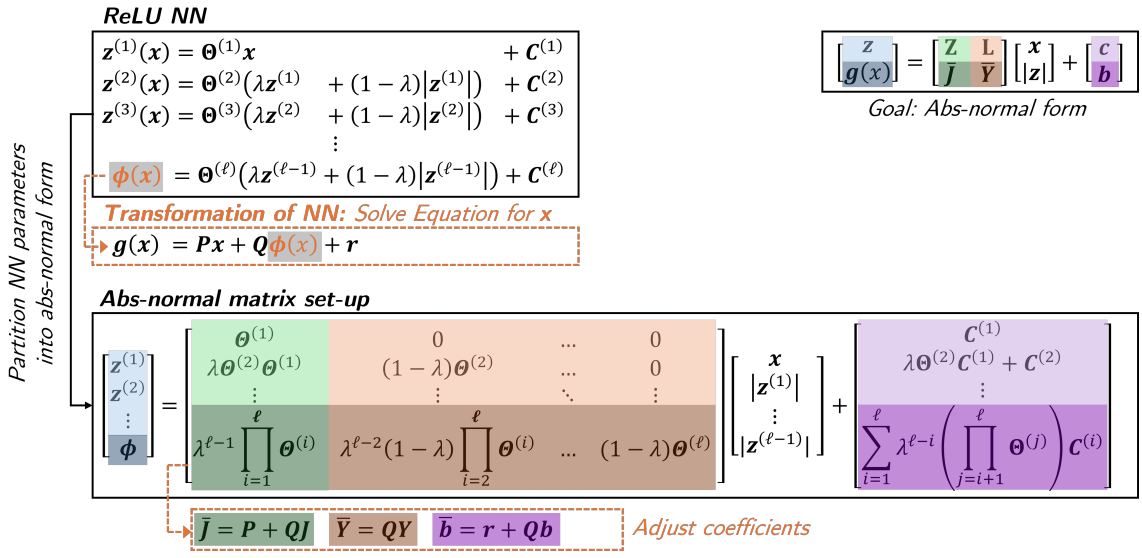
$$\overline{\mathbf{J}} := \mathbf{P} + \mathbf{Q}\mathbf{J} \qquad \overline{\mathbf{Y}} =: \mathbf{Q}\mathbf{Y} \qquad \overline{\mathbf{b}} =: \mathbf{r} + \mathbf{Q}\mathbf{b}$$

3. Solve for $\mathbf{x}$ using complementarity approach.

Root finding for ReLU neural networks

(In preparation)

Transformations of NN to abs-normal form



Solving neural ODEs

(In preparation)

Linear multistep methods to solve first order neural ODEs

- Linear multistep methods solves root finding problems at each time step.
  - **Backward Differentiation**: $BDF_s: \sum_{k=0}^{s} a_k x_{n+k} - h\beta_s f(t_{n+s}, x_{n+s}) = 0$, where $x' = f(t, x)$
  - **Adams-Moulton**: $AM_s: x_{s+1} - x_s - h\sum_{k=0}^{s} \beta_k f(t_{n+1-k}, x_{n+1-k}) = 0$, where $x' = f(t, x)$
- Coefficients for s-steps of multistep methods:

Method	Order	**P**	**Q**	**r**
BDF	1	**I**	$h\mathbf{I}$	$-x^{(0)}$
	2	**I**	$\frac{2}{3}h$	$-\frac{4}{3}x^{(1)} + \frac{1}{3}x^{(0)}$
	...			
	s	**I**	$-\frac{1}{a_s}h\mathbf{I}$	$\sum_{k=0}^{s-1} -a_k x^{(n+k)}$
AM	1	**I**	$h\mathbf{I}$	$-x^{(0)}$
	2	**I**	$-\frac{1}{2}h\mathbf{I}$	$-x^{(1)} - \frac{1}{2}hf(t^{(1)}, x^{(1)})$
	...			
	s	**I**	$-\frac{1}{a_s}h\mathbf{I}$	$-x_{n+s-1} - h\sum_{k=1}^{s-1} \beta_k f(t_{n+k}, x_{n+k})$

New root-finding approach

Zhang & Khan, *AD 2024*

New MLCP formulation

- Since $\mathbf{L}$ is strictly lower triangular, $(\mathbf{I}-\mathbf{L})$ is unit lower triangular and invertible
- Define matrices and vectors:

$$\tilde{\mathbf{c}} := (\mathbf{I}-\mathbf{L})^{-1}\mathbf{c}, \qquad \tilde{\mathbf{b}} := \mathbf{b}+\mathbf{Y}\tilde{\mathbf{c}}, \qquad \tilde{\mathbf{L}} := (\mathbf{I}-\mathbf{L})^{-1}(\mathbf{I}+\mathbf{L}),$$
$$\tilde{\mathbf{Y}} := \mathbf{Y}(\mathbf{I}+\tilde{\mathbf{L}}), \qquad \tilde{\mathbf{Z}} := (\mathbf{I}-\mathbf{L})^{-1}\mathbf{Z}, \qquad \tilde{\mathbf{J}} := \mathbf{J}+\mathbf{Y}\tilde{\mathbf{Z}}.$$

- $\mathbf{f}(\mathbf{x})=\mathbf{0}$ solves the following MLCP:

$$\begin{aligned} \mathbf{0} &= \tilde{\mathbf{b}}+\tilde{\mathbf{J}}\mathbf{x}+\tilde{\mathbf{Y}}\mathbf{w} \\ \mathbf{0}\le\mathbf{w} &\perp \tilde{\mathbf{c}}+\tilde{\mathbf{Z}}\mathbf{x}+\tilde{\mathbf{L}}\mathbf{w}\ge\mathbf{0}. \end{aligned}$$

where $\mathbf{w}\in\mathbb{R}^s := \max(\mathbf{0},-\mathbf{z})$.

Properties

- Fewer variables, and fewer equations than previous MLCP (per Griewank et. al, 2015)
- No requirement that $m=n$ or nonsingular $\tilde{\mathbf{J}}$
- Compatible with PATH solver

New root-finding approach

Zhang & Khan, *AD 2024*

New LCP formulation

- If $m = n$ and $\tilde{\mathbf{J}}$ is nonsingular, define:

$$\sigma := \tilde{\mathbf{c}} - \tilde{\mathbf{Z}}\tilde{\mathbf{J}}^{-1}\tilde{\mathbf{b}}, \qquad \rho := \tilde{\mathbf{L}} - \tilde{\mathbf{Z}}\tilde{\mathbf{J}}^{-1}\tilde{\mathbf{Y}}.$$

- $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ can be obtained via:

$$\tilde{\mathbf{J}}\mathbf{x} = -\tilde{\mathbf{b}} - \tilde{\mathbf{Y}}\mathbf{w},$$

  where $\mathbf{w} \in \mathbb{R}^s$ solves the LCP:

$$\mathbf{0} \leq \mathbf{w} \perp \sigma + \rho\mathbf{w} \geq \mathbf{0}.$$

Properties

- From our experience, $\tilde{\mathbf{J}}$ seems to be nonsingular when each $|z_i|$ contributes meaningfully to f
- Fewer variables than both MLCP formulations
- More suited to PATH solver

New optimization approach

Zhang & Khan, *AD 2024*

- Consider a piecewise-affine function $f : \mathbb{R}^n \to \mathbb{R}$ in abs-normal form:

$$\begin{bmatrix} \mathbf{z} \\ f(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \mathbf{c} \\ \mathbf{b} \end{bmatrix} + \begin{bmatrix} \mathbf{Z} & \mathbf{L} \\ \mathbf{J} & \mathbf{Y} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ |\mathbf{z}| \end{bmatrix}.$$

- Minimizing $f(\mathbf{x})$ is equivalent to solving a linear program with complementarity constraints (LPCC):

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^n, \mathbf{w}\in\mathbb{R}^s} \quad & \tilde{b} + \tilde{\mathbf{J}}\mathbf{x} + \tilde{\mathbf{Y}}\mathbf{w} \\ \text{s.t.} \quad & \mathbf{0} \leq \mathbf{w} \perp \tilde{\mathbf{c}} + \tilde{\mathbf{Z}}\mathbf{x} + \tilde{\mathbf{L}}\mathbf{w} \geq \mathbf{0}. \end{aligned}$$

  - can be solved using Branch-and-Cut [Yu et al. *Math. Program. Comput. (2019)*]

- If bounds on $\mathbf{x}^\star$ and $\mathbf{w}^\star$ are known, we can rewrite LPCC as a MILP using "big-M":

$$\begin{aligned} \min_{\mathbf{x},\mathbf{w},\mathbf{y}} \quad & \tilde{b} + \tilde{\mathbf{J}}\mathbf{x} + \tilde{\mathbf{Y}}\mathbf{w} \\ \text{s.t.} \quad & \mathbf{0} \leq \mathbf{w} \leq \mu\mathbf{y}, \\ & \mathbf{0} \leq \tilde{\mathbf{c}} + \tilde{\mathbf{Z}}\mathbf{x} + \tilde{\mathbf{L}}\mathbf{w} \leq \mu(\mathbf{e} - \mathbf{y}), \\ & \mathbf{y} \in \{0,1\}^s. \end{aligned}$$

Background: horizon function

Horizon function [Dontchev and Rockafellar, *Set-Valued Analysis* (2004)]

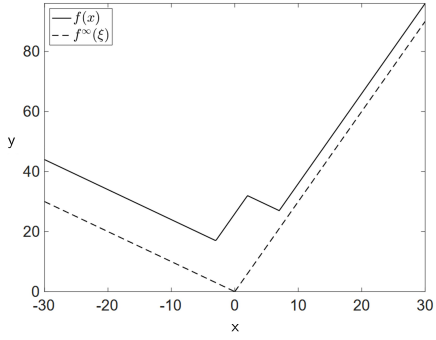
Given a continuous function $f : \mathbb{R}^n \to \mathbb{R}$, the horizon function for f is the mapping:

$$f^\infty(\boldsymbol{\xi}) = \lim_{\delta \to 0^+} \left(\inf\{\lambda f(\tfrac{\mathbf{x}}{\lambda}) : \quad 0 < \lambda < \delta, \quad \|\mathbf{x} - \boldsymbol{\xi}\| < \delta\} \right).$$

Example

The horizon function $f^\infty(x)$ of the following piecewise-affine function:

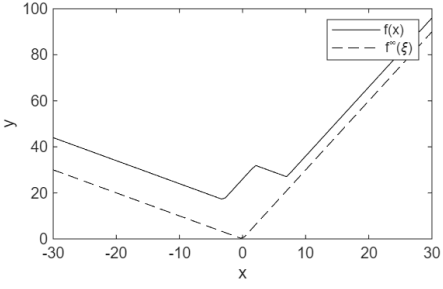
$$f(x) = x + \Big|2|x-2| - 10\Big| + 20,$$



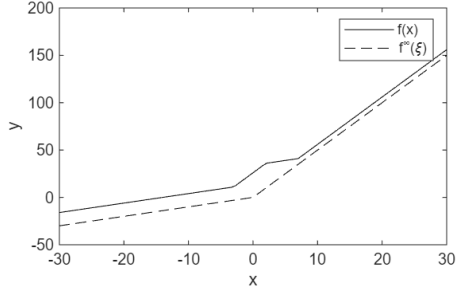
Existence of minima

Zhang & Khan, *AD 2024*

- Any horizon function f^∞ has $\inf_{\xi \in \mathbb{R}^n} f^\infty(\xi) \in \{-\infty, 0\}$.
- A global minimum exists if and only if $\inf_{\xi \in X} f^\infty(\xi) = 0$,
- Then, if and only if $f^\infty(\xi) + 1 = 0$ has no solution $\xi \in \mathbb{R}^n$, a global minimum exists.



Function with global minimum.



Function without global minimum.

Existence of minima

Zhang & Khan, *AD 2024*

- How to solve $f^\infty(\xi) + 1 = 0$?
- The horizon function f^∞ is piecewise-linear, and has the abs-normal form:

$$\begin{bmatrix} \zeta \\ f^\infty(\xi) \end{bmatrix} \equiv \begin{bmatrix} \mathbf{Z} & \mathbf{L} \\ \mathbf{J} & \mathbf{Y} \end{bmatrix} \begin{bmatrix} \xi \\ |\zeta| \end{bmatrix},$$

  where ζ/ξ plays the same role as $\mathbf{z}/\mathbf{x}$ did.
- The following MLCP solves $f^\infty(\xi) + 1 = 0$:

$$\begin{aligned} 0 &= 1 + \tilde{\mathbf{J}}\xi + \tilde{\mathbf{Y}}\omega \\ \mathbf{0} \le \omega &\perp \tilde{\mathbf{Z}}\xi + \tilde{\mathbf{L}}\omega \ge \mathbf{0}, \end{aligned}$$

  where $\omega \in \mathbb{R}^s := \max(\mathbf{0}, -\zeta)$.
- MLCP characterizes existence of a global minimum for piecewise-affine functions.
- This enables verification of a global minimum before minimization.

Implementation and examples

Zhang & Khan, *AD 2024*

- Implemented in Julia v1.10.5
  - MLCP and LCP solvers: PATHSolver.jl 1.7.5 and BARON.jl v0.8.3 in JuMP
  - Input: four matrices and two vectors of the abs-normal form

General root-finding example

- Consider a piecewise-affine function:

$$f_1(x_1, x_2) = \Big||x_1 + 2| + x_2 - 1\Big| - x_2 - 1,$$
$$f_2(x_1, x_2) = |x_1 + 2| + 2x_2 - 1,$$

  with the abs-normal form:

$$\begin{bmatrix} z_1 \\ z_2 \\ f_1 \\ f_2 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \\ -1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & -1 & 0 & 1 \\ 0 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ |z_1| \\ |z_2| \end{bmatrix}.$$

- The root $(x_1, x_2) = (0, -0.5)$ was found using both the MLCP and the LCP formulations.

Implementation and examples

Zhang & Khan, *AD 2024*

- Implemented in Julia v1.10.5
  - MLCP solver for verification: BARON.jl v1.7.5 in JuMP
  - MILP and LPCC solver for minimization: BARON.jl v1.7.5 in JuMP
  - Input: four matrices and two vectors of the abs-normal form

General optimization example

- Objective:

$$f(x) = \Big|x_1 + |2x_2 - 1|\Big| + |3 + x_3|\,,$$

with the abs-normal form:

$$\begin{bmatrix} z_1 \\ z_2 \\ z_3 \\ f(x) \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 2 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ |z_1| \\ |z_2| \\ |z_3| \end{bmatrix}.$$

- The existence of a global minimum for $f(\mathbf{x})$ was verified.
- The minimum is $\mathbf{x}^* = (0.0, 0.5, -3.0)$.

Implementation and examples

(In preparation)

- Implemented in Julia v1.10.5
  - MLCP and LCP solvers: PATHSolver.jl 1.7.5 in JuMP
  - Input: four matrices and two vectors of the abs-normal form

Case 1: Root of a Neural Net

- To calculate the root of the neural net - i.e., solve $h(x, \phi) = 0$.
- Set $\mathbf{P} = \mathbf{0}$, $\mathbf{Q} = \mathbf{I}$, and $\mathbf{r} = \mathbf{0}$.
- Consider $\mathbf{f}(\mathbf{X}) = (\mathbf{A}\mathbf{X}^T - \mathbf{b})^T$ where:

$$\mathbf{A} = \begin{bmatrix} 2 & -3 & 1 \\ -4 & 1 & -2 \\ 5 & -3 & 9 \end{bmatrix}, \mathbf{b} = \begin{bmatrix} 1 \\ 4 \\ -2 \end{bmatrix}$$

A 1-layer NN mapping of $\mathbf{f}(\mathbf{X})$ produces a local solution of the root.

$$\mathbf{x} = \begin{bmatrix} -0.276 & -0.244 & -0.019 \end{bmatrix}^T$$

Implementation and examples

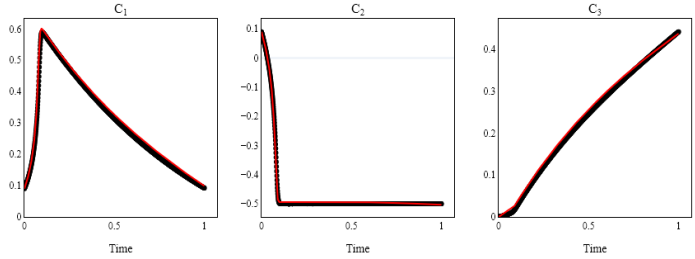
(In preparation)

Case 2: Trajectory of neural ODEs

- Consider the following CSTR ODE whose dynamics are represented using a neural network where $C(0) = \begin{bmatrix} 0.09 & 0.09 & 0.00 \end{bmatrix}^T$, $t \in [0, 1]$.

$$\begin{bmatrix} \dot{C}_1 \\ \dot{C}_2 \\ \dot{C}_3 \end{bmatrix} = \begin{bmatrix} -C_1 - k_1 + (C_2 + k_2) \exp\left(\frac{k_3 C_1}{C_1 + 2}\right) \\ -C_2 - k_2 - (C_2 + k_2) \exp\left(\frac{k_3 C_1}{C_1 + 2}\right) \\ C_1^2 + C_2^2 + k_4 \end{bmatrix}$$

- *Solving the BDF ($s = 5$) root-finding problem at each time step produces:*



Implementation and examples

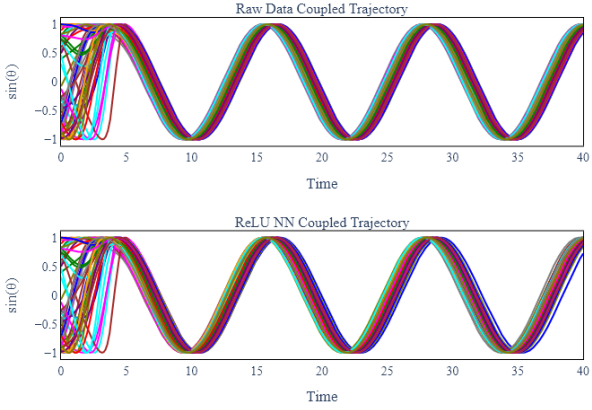
(In preparation)

- The Kuramoto model describes large sets of coupled phase oscillators that synchronize with each other when interacting.
- Represented as a continuous-time ODE where ω_i is unit i's natural frequency and K is the coupling strength.

$$\dot{\theta}_i = \omega_i - K\boldsymbol{r}(t)\sin(\theta_j - \theta_i)$$

$$\boldsymbol{r}(t)e^{i\psi(t)} = \frac{1}{N}\sum_{j}^{N} e^{i\theta_j(t)}$$

- Consider a model with $N = 50$ oscillators and a timestep of $6 \cdot 10^{-4}$.
- Solving the root-finding problem at each time step using Euler produces:



Root-finding scaling experiment

Zhang & Khan, *AD 2024*

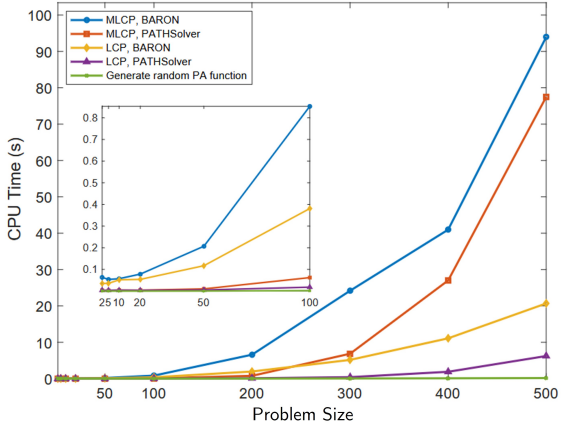
- Consider a randomly generated piecewise-affine function:

$$f : \mathbb{R}^n \to \mathbb{R}^n$$

in abs-normal form with $s = n$, where the coefficients

  - **b**, **c**, **Y** are generated based on a normal distribution and rounded to the nearest integers
  - **Z** is identity matrix; **J** is zero matrix
  - **L** has ones on the first subdiagonal, with all other elements being zero

- The average CPU time (s) for solving $\mathbf{f}(\mathbf{x}) = \mathbf{0}$:
  - PATH may fail to solve MLCP and LCP at larger n, BARON does not
  - LCP outperforms MLCP; PATH outperforms BARON



Root-finding scaling experiment

Zhang & Khan, *AD 2024*
Comparison with approach of Griewank et al. (2015)

- Consider a randomly generated piecewise-affine function:

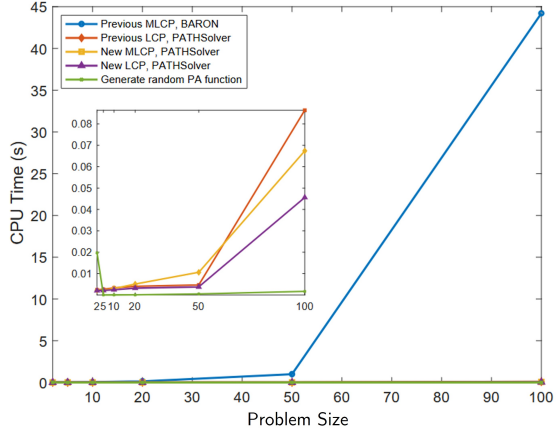
$$f : \mathbb{R}^n \to \mathbb{R}^n$$

in abs-normal form, where the coefficients

  - $\mathbf{b}, \mathbf{c}, \mathbf{Y}$ are generated based on a normal distribution and rounded to the nearest integers
  - $\mathbf{Z}, \mathbf{J}$ are identity matrices; $\mathbf{J}$ has to be nonsingular
  - $\mathbf{L}$ has ones on the first subdiagonal, with all other elements being zero

- Griewank et al.'s MLCP can not be solved by PATH; BARON is used

- The average CPU time (s) for solving $\mathbf{f}(\mathbf{x}) = \mathbf{0}$:
  - PATH may fail to solve $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ when $n \geq 100$
  - For the same problem, our new LCP requires less CPU time



Optimization scaling experiment

Zhang & Khan, *AD 2024*

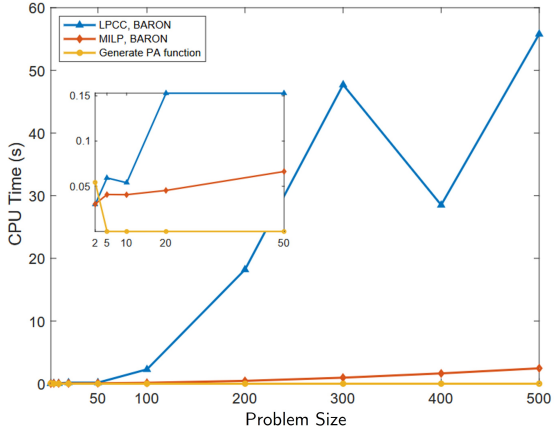
- Consider a piecewise-affine function $f : \mathbb{R}^n \to \mathbb{R}$:

$$f(\mathbf{x}) = \Big|\big||1000x_1| + 1000x_2\big| + \cdots + 1000x_n\Big| + 1.0.$$

in abs-normal form, where

  - $s = n$,
  - the global minimum existence is verified using MLCP formulation
  - before minimizing using MILP and LPCC formulations.

- The average CPU time (s) for minimizing $f(\mathbf{x})$:
  - MILP is faster than LPCC for all n



Computational Complexity

Zhang & Khan, *AD 2024*

- Setting up the LCP scales with polynomial complexity
  - Coefficient definition includes basic algebra steps - i.e., matrix multiplication and solving linear systems
- Solving LCP is computationally expensive
  - Comparable to integer linear programs
  - Preserve continuity when there's a discrete switch

Conclusions

- Piecewise-affine approximations for nonsmooth functions are useful and can be constructed using a forward-AD like procedure
- A piecewise-affine function can be represented in abs-normal form, which can be constructed using a forward-AD like procedure
- New root-finding approach for piecewise-affine functions, with fewer variables and equations
- New optimization formulations for piecewise-affine functions
- Verification of a global minimum can be performed before minimization
- New root-finding approaches for functions with embedded ReLU neural networks
- Proof-of-concept implementations developed in Julia
- Code available on GitHub:
  `https://github.com/kamilkhanlab/abs-normal`
- Funding: MACC, NSERC, ORF-RE